



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table><tr><td>Student 1</td><td>Student 2</td></tr><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></table> Name 1: LEE YIN SHEN Metric No: A23CS0236 Name 2: BRENDAN CHIA YAN FEI Metric No: A23CS0211	Student 1	Student 2	<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>
Student 1	Student 2				
<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>				
SUBMISSION DATE:	_____				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

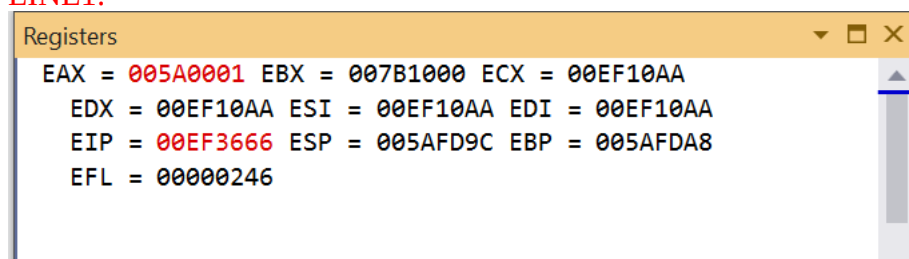
a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX
LINE3	AX = 0009h BX = 0001h	Exchange the value between register AX and BX
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX into variable var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register BX into variable var2

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:

```
Registers
EAX = 005A0001 EBX = 007B0009 ECX = 00EF10AA
EDX = 00EF10AA ESI = 00EF10AA EDI = 00EF10AA
EIP = 00EF366D ESP = 005AFD9C EBP = 005AFDA8
EFL = 00000246
```

LINE3:

```
Registers
EAX = 005A0009 EBX = 007B0001 ECX = 00EF10AA
EDX = 00EF10AA ESI = 00EF10AA EDI = 00EF10AA
EIP = 00EF366F ESP = 005AFD9C EBP = 005AFDA8
EFL = 00000246
```

LINE4:

```
Registers
EAX = 005A0009 EBX = 007B0001 ECX = 00EF10AA
EDX = 00EF10AA ESI = 00EF10AA EDI = 00EF10AA
EIP = 00EF3675 ESP = 005AFD9C EBP = 005AFDA8
EFL = 00000246
```

LINE5:

```
Registers
EAX = 005A0009 EBX = 007B0001 ECX = 00EF10AA
EDX = 00EF10AA ESI = 00EF10AA EDI = 00EF10AA
EIP = 00EF367C ESP = 005AFD9C EBP = 005AFDA8
EFL = 00000246
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx       ; LINE5
    inc eax           ; LINE6
    mov Rval,eax      ; LINE7
    exit
main endp
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFE6h	Negate register EAX with 2's complement
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFFF6h Zval = 00000028h	Subtract value of Zval (40d) from register EBX
LINE5	EAX = FFFFFFFF0Ch EBX = FFFFFFFF6h	Add value of register EBX to register EAX
LINE6	EAX = FFFFFFFDDh	Increment of EAX by 1
LINE7	EAX = FFFFFFFDDh Rval = FFFFFFFDDh	Move value stored in EAX to variable Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 0000001A EBX = 00B22000 ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 00881015 ESP = 0084FBD8
EBP = 0084FBE4 EFL = 00000246
```

LINE2:

```
Registers
EAX = FFFFFFFE6 EBX = 00C90000 ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 00881017 ESP = 00F8FC90
EBP = 00F8FC9C EFL = 00000293

0x00884008 = 0000001E
```

LINE3:

```
Registers
EAX = FFFFFFFE6 EBX = 0000001E ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 0088101D ESP = 0136F900
EBP = 0136F90C EFL = 00000293

0x0088400C = 00000028
```

LINE4:

```
Registers
EAX = FFFFFFFE6 EBX = FFFFFFF6 ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 00881023 ESP = 00F8FC90
EBP = 00F8FC9C EFL = 00000287
```

LINE5:

```
Registers
EAX = FFFFFFFDC EBX = FFFFFFF6 ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 00881025 ESP = 00F8FC90
EBP = 00F8FC9C EFL = 00000283
```

LINE6:

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFF6 ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 00881026 ESP = 00F8FC90
EBP = 00F8FC9C EFL = 00000287

0x00884000 = 00000000
```

LINE7:

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFF6 ECX = 00881005 EDX = 00881005
ESI = 00881005 EDI = 00881005 EIP = 0088102B ESP = 00F8FC90
EBP = 00F8FC9C EFL = 00000287
```

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2             ; LINE2
    add eax, var3        ; LINE3
    dec eax              ; LINE4
    exit
main endp
end main
```

Answer Q3

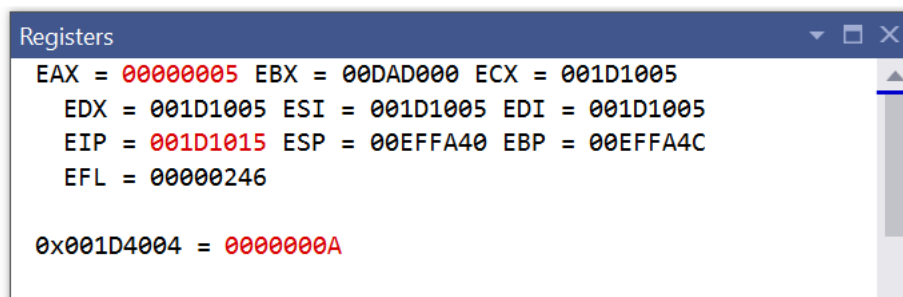
a) Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiply the value of var2 (10d) to register EAX
LINE3	EAX = 00000046h var3 = 00000014h	Add the value of var3 (20d) to register EAX
LINE4	EAX = 00000045h var4 = 00000045h	Move the value of register EAX into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:

```
Registers
EAX = 00000032 EBX = 00DAD000 ECX = 001D1005
EDX = 00000000 ESI = 001D1005 EDI = 001D1005
EIP = 001D101B ESP = 00EFFA40 EBP = 00EFFA4C
EFL = 00000202

0x001D4008 = 00000014
```

LINE3:

```
Registers
EAX = 00000046 EBX = 00DAD000 ECX = 001D1005
EDX = 00000000 ESI = 001D1005 EDI = 001D1005
EIP = 001D1021 ESP = 00EFFA40 EBP = 00EFFA4C
EFL = 00000202
```

LINE4:

```
Registers
EAX = 00000045 EBX = 00DAD000 ECX = 001D1005
EDX = 00000000 ESI = 001D1005 EDI = 001D1005
EIP = 001D1022 ESP = 00EFFA40 EBP = 00EFFA4C
EFL = 00000202
```

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move the immediate value (5d) into register BX
LINE3	AX = 00C8h BX = 0005h	Multiply the value of register BX to register AX and store the value in register DX:AX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2 (10d) into register BX
LINE5	BX = 0007h	Subtract the value in register BX with an immediate value (3d)
LINE6	AX = 001Ch BX = 0007h DX = 0004h	Divide the register AX by register BX, the quotient is then store in register AX and the remainder store in register DX
LINE7	AX = 001Ch var4 = 001Ch	Move the value of register AX into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers	
EAX = 01150028	EBX = 00E67000 ECX = 00D31005 EDX = 00D31005
ESI = 00D31005	EDI = 00D31005 EIP = 00D31016 ESP = 0115FC28
EBP = 0115FC34	EFL = 00000246

LINE2:

Registers	
EAX = 01150028	EBX = 00E60005 ECX = 00D31005 EDX = 00D31005
ESI = 00D31005	EDI = 00D31005 EIP = 00D3101A ESP = 0115FC28
EBP = 0115FC34	EFL = 00000246

LINE3:

Registers	
EAX = 011500C8	EBX = 00E60005 ECX = 00D31005 EDX = 00D30000
ESI = 00D31005	EDI = 00D31005 EIP = 00D3101D ESP = 0115FC28
EBP = 0115FC34	EFL = 00000202

LINE4:

Registers	
EAX = 011500C8	EBX = 00E6000A ECX = 00D31005 EDX = 00D30000
ESI = 00D31005	EDI = 00D31005 EIP = 00D31024 ESP = 0115FC28
EBP = 0115FC34	EFL = 00000202

LINE5:

Registers	
EAX = 011500C8	EBX = 00E60007 ECX = 00D31005 EDX = 00D30000
ESI = 00D31005	EDI = 00D31005 EIP = 00D31028 ESP = 0115FC28
EBP = 0115FC34	EFL = 00000202

LINE6:

Registers	
EAX = 0115001C	EBX = 00E60007 ECX = 00D31005 EDX = 00D30004
ESI = 00D31005	EDI = 00D31005 EIP = 00D3102B ESP = 0115FC28
EBP = 0115FC34	EFL = 00000202

LINE7:

Registers	
EAX = 0115001C	EBX = 00E60007 ECX = 00D31005 EDX = 00D30004
ESI = 00D31005	EDI = 00D31005 EIP = 00D31031 ESP = 0115FC28
EBP = 0115FC34	EFL = 00000202

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

a) Write a full program to execute the Code Snippet 1.

b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX,0           ; Clear DX
MOV AX,1000h        ; Load 1000h into AX
MOV CX, 25h         ; Load 25h into CX
MUL CX              ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

a) Screenshot of full program (.asm) :

```
1  TITLE Lab2
2
3  ; Authors: LEE YIN SHEN& BRENDAN CHIA YAN FEI
4  ; Date: 19 / 5 / 2024
5  ; Question: 5
6
7  include irvine32.inc
8
9  .data
10
11 .code
12 main proc
13
14     mov dx, 0; Clear DX
15     mov ax, 1000h; Load 1000h into AX
16     mov cx, 25h; Load 25h into CX
17     mul cx; Multiply AX by CX, storing the result in DX : AX
18     call DumpRegs
19
20     exit; Exit the program
21
22 main endp
23 end main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=010F5000  EBX=00E67000  ECX=00480025  EDX=00480002
ESI=004810AA  EDI=004810AA  EBP=010FF934  ESP=010FF928
EIP=00483674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1
```

Q6. Given the following instructions as is Code Snippet 2.

a) Write a full program to execute the Code Snippet 2.

b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

a) Screenshot of full program (.asm) :

```
1  TITLE Lab2
2
3  ; Authors: LEE YIN SHEN& BRENDAN CHIA YAN FEI
4  ; Date: 19 / 5 / 2024
5  ; Question: 6
6
7  include irvine32.inc
8
9  .data
10
11 .code
12 main proc
13
14     mov dx, 0; Clear DX to form the 16 - bit dividend in DX : AX
15     mov ax, 803h; Load the dividend(8003h) into AX
16     mov bx, 10h; Load the divisor(10h) into BX
17     div bx; Divide DX : AX by BX, AX = quotient & DX = remainder
18
19     call DumpRegs; Display the contents of registers
20
21     exit; Exit the program
22
23 main endp
24 end main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=008F0080 EBX=00710010 ECX=00A110AA EDX=00A10003
ESI=00A110AA EDI=00A110AA EBP=008FF824 ESP=008FF818
EIP=00A13674 EFL=00000246 CF=0 SF=0 ZF=1 OF=0 AF=0 PF=1
```